Arch-Plotter Reference Manual

Auto-generated Documentation by using tidy library

Version 0.1.0

Contents

Introduction & Setup 3
Unit Systems 3
Tracing Walls 3
- Movement Commands 4
- Jumps & Marks (Lifting the Pen) 4

Architectural Components 4
- Doors and Openings 4
- Windows 5
- Stairs 5
- Columns 5

Smart Rooms & O.T.S. 5
Drafting Utilities 6
Architectural Components 6
- `door()` 6
- double-door() 7
- window() 7
- opening() 8
- stairs() 8
- column() 9

Smart Rooms & Zones 9
- poly-room() 9
- ots() (Open To Sky / Shaft) . 10

Land Surveying Engine (Plotter module) 11
- trace-plot() 11
- draw-plot() 11
- Automated CAD Schedules 12

Arch (API) 12
- A 12
- C 12
- D 13
- DropX 13
- DropY 14
- GoToMark 14
- Home 15
- JA 15
- JD 16
- JDropX 16
- JDropY 16
- JL 17
- JR 17
- JU 18
- JumpToMark 18
- L 18
- Mark 19
- R 19
- U 20
- arch-canvas 20
- callout 21
- cm 22

column 22
create-grid 23
dim 24
door 25
double-door 26
drafting-grid 27
drafting-ruler 28
draw-walls 29
format-area 30
format-dist 30
ft 31
get-align-dy 31
get-angle 32
get-area 32
get-length 33
get-pts 33
i-dim 34
inset-polygon 35
intersect-lines 35
m 36
move 36
opening 37
ots 39
poly-room 40
print-marks 41
stairs 42
trace-walls 43
wall-shape 44
window 45
    bricks-fill 46
    hatch-fill 46
    grass-fill 46

Introduction & Setup

The `Arch.typ` engine is designed to rapidly draft floor plans, generate dynamic walls with clean intersections, and drop in parametric architectural components (doors, windows, stairs, furniture) using intuitive commands.

To use the Architecture engine, import the `arch-canvas` wrapper from the main library, and then import the `Arch.typ` module directly from the `src/` folder to access the shorthand commands without prefixes.

```
#import "@preview/arch-plotter:0.1.0": *

#arch-canvas(scale: 0.5cm, {
  // Your drafting code here
})
```

Unit Systems

The engine operates natively in Decimal Feet. However, helper functions are provided so you can draft using standard Architectural notation or Metric units.

`ft(feet, inches)`: Converts feet and inches to decimal feet. E.g., ft(5, 6) returns 5.5.

m(meters): Converts meters to decimal feet. E.g., m(2) returns 6.56.

cm(centimeters): Converts centimeters to decimal feet.

Formatting Distances: Use format-dist(value, unit-system: "imperial") or unit-system: "metric" to automatically format raw numbers into beautiful text (e.g., 5′6 1/2″ or 1.5 m).

Tracing Walls

The core of the Architecture engine is the trace-walls() function. It acts like a continuous pen on paper.

Code snippet

```
let house = trace-walls(start: (0,0), (
  R(20), U(15), L(20), C()
))
draw-walls(house, default-thickness: 0.75)
```

Movement Commands

R(len) / L(len): Move Right / Left.

U(len) / D(len): Move Up / Down.

A(angle, len): Move at a specific angle.

DropX(p1, p2) / DropY(p1, p2): Raycast your current line horizontally or vertically until it intersects with a target boundary (like an angled wall).

C() or Home(): Close the shape back to the start.

Jumps & Marks (Lifting the Pen)

If you need to move the cursor without drawing a wall, use Jump commands:

JR(len), JL(len), JU(len), JD(len): Jump Right, Left, Up, Down.

JA(angle, len): Jump at an angle.

Mark("name"): Save the current coordinate in the engine's memory.

GoToMark("name"): Draw a wall directly to a saved mark.

JumpToMark("name"): Lift the pen and teleport to a saved mark.

Architectural Components

Components snap relative to the coordinates you trace.

Doors and Openings

Code snippet

```
door(from, to, dist, width: 3.0, hinge: "left", swing:
"in")
```

```
double-door(from, to, dist, width: 6.0, swing: "out")
opening(from, to, dist, width: 4.0, style: "cross")
```

from, to: The two coordinates representing the wall segment.

dist: How far along that wall (from the from point) to place the door.

hinge / swing: Controls the exact open position of the door leaf.

Windows

Code snippet window(from, to, dist, width: 4.0, thickness: 0.75) Automatically erases the wall behind it and draws the glass panes and frame caps.

Stairs

Code snippet stairs((10, 10), steps: (5, 5), width: 3.0, run: 0.75, turn: “right”) Generates a parametric staircase.

steps: Can be a single integer (e.g., 12) for straight stairs, or an array (e.g., (5, 5)) to automatically insert landings.

turn: “left” or “right” determines which way the landing rotates.

Columns

Code snippet column((5, 5), size: 1.0, sides: 4) column((10, 5), size: 1.0, sides: 30)

Smart Rooms & O.T.S.

If you don’t want to manually trace every wall, you can pass an array of coordinates to generate instant spaces.

Code snippet

```
let pts = ((0,0), (10,0), (10,10), (0,10))
poly-room(pts, "Master Bed", wall-widths: (0.75,), show-
dim: true, show-area: true)
ots(pts, name: "Shaft")
```

wall-widths: Pass an array of thicknesses. The engine will automatically calculate the complex inset geometry, shift the walls inward, and calculate the true internal area.

Drafting Utilities

```
dim(from, to): Standard external dimension line.

i-dim(from, to): Internal dimension line (automatically
shrinks to fit between wall thicknesses).

callout(target, text-pos, label): Draws a leader line with
a shoulder to point at specific details.

drafting-grid(width, height): Overlays a faint measurement
grid.

drafting-ruler(width, height)
```

: Draws CAD-style coordinate rulers on the left and top edges.

Architectural Components

Components are designed to snap seamlessly to your traced walls. For components that attach to a wall (like doors and windows), you provide the `from` and `to` coordinates of the wall segment, and the `dist` (distance) from the start point to place the component.

`door()`

Places a parametric single door along a wall segment. It automatically calculates rotation and draws the door frame, leaf, and swing arc.

Parameters:

- `from` (*coordinate*): The starting point of the wall segment (e.g., `(0,0)`).
- `to` (*coordinate*): The ending point of the wall segment.
- `dist` (*float*): Distance from the `from` point to the center of the door.
- `width` (*float*) - Default: `3.0`: The width of the door opening.

- `hinge` (*string*) - Default: `"left"`: Which side the hinge is on. Values: `"left"`, `"right"`.
- `swing` (*string*) - Default: `"in"`: Which way the door opens. Values: `"in"`, `"out"`.
- `open` (*angle*) - Default: `90deg`: The angle the door leaf is open.
- `thickness` (*float*) - Default: `0.75`: Thickness of the wall to match.
- `label` (*content*) - Default: `none`: Optional text to display on the door.

Example:

```
door((0,0), (10,0), 5.0, width: 3.0, hinge: "right",
swing: "out")
```

double-door()

Places a double-leaf door. Functions identically to door(), but splits the width into two sweeping arcs.

Parameters:

Shares all parameters with door() except hinge (as it has hinges on both sides).

window()

Places a window along a wall. It automatically erases the wall lines behind it and draws the glass panes and frame caps.

Parameters:

wall-start *(coordinate)*: The starting point of the wall.

wall-end *(coordinate)*: The ending point of the wall.

dist *(float)*: Distance from the start point to the center of the window.

width *(float)* - Default: 1.0: The width of the window.

thickness *(float)* - Default: 0.75: The thickness of the wall.

label *(string|content)* - Default: none: Optional text (e.g., “W1”).

label-offset *(float)* - Default: 0.8: Distance to shift the label away from the glass.

Example:

```
window((0,0), (0,10), 5.0, width: 4.0, label: "W-1")
```

opening()

Creates a cased opening or pass-through in a wall without a door leaf.

Parameters:

from, to, dist, width, thickness: Same as door().

style (string) - Default: “empty”: The visual style of the opening. Values: “empty”, “line”, “rect”, “cross” (often used for shafts or hidden lines).

stairs()

Generates a parametric staircase. Can create straight runs, L-shapes, or U-shapes automatically based on the turns array.

Parameters:

pos (coordinate): The starting bottom-left coordinate of the stairs.

steps (int | array) - Default: 12: The number of treads. If an array is passed (e.g., (5, 5)), it automatically generates a landing between the runs.

width (float) - Default: 3.0: The width of the staircase.

run (float) - Default: 0.75: The depth of each individual tread.

angle (angle) - Default: 0deg: The global rotation of the entire staircase.

turn (string | array) - Default: “right”: The direction the landing turns. Values: “right”, “left”.

split-landing (boolean) - Default: false: If true, draws a diagonal line across the landing to indicate a winder step.

Example:

Code snippet stairs((10, 10), steps: (6, 6), turn: “left”, width: 3.5)

column()

Draws a structural column (square, circular, or polygonal).

Parameters:

pos (coordinate): The center or edge coordinate of the column.

size (float) - Default: 0.75: The overall width/diameter.

sides (int) - Default: 4: Number of sides. 4 = Square, 6 = Hexagon, 30+ = Perfect Circle.

Smart Rooms & Zones

poly-room()

Generates a complete, dimensioned room from a set of perimeter points. If wall widths are provided, it automatically calculates the complex inset geometry and shrinks the inner area.

Parameters:

pts (array): An array of coordinates (e.g., ((0,0), (10,0), (10,10), (0,10))).

name (string): The room label (e.g., “Master Bedroom”).

wall-widths (array|none) - Default: none: An array of wall thicknesses. E.g., (0.75, 0.5).

show-area (boolean) - Default: true: Displays the calculated square footage.

show-dim (boolean) - Default: true: Displays the overall Width x Height.

units (string) - Default: “imperial”: Formatting system. Values: “imperial”, “metric”.

ots() (Open To Sky / Shaft)

Draws a designated shaft or open area with a standard architectural “X” crossed through it.

Parameters:

Shares parameters with poly-room(), but defaults show-area and show-dim to false.

Land Surveying Engine (Plotter module)

The Plotter engine uses the exact same tracing logic as the Architecture engine, but is optimized for massive coordinates, angle properties, layer management, and CAD schedules.

trace-plot()

Traces a property boundary. Uses the exact same movement commands (R, L, U, D, Mark, etc.) as trace-walls.

Special Plotter Command:

Layer("name"): Switches the active drawing layer. Any lines drawn after this command are visual only and are ignored by the Area and Perimeter calculators.

draw-plot()

Renders the output of trace-plot with survey-specific visual data.

Parameters:

plot-data (dictionary): The data returned from trace-plot.

name (string) - Default: "": The plot identifier (e.g., "Plot 12").

show-dim (boolean) - Default: false: Automatically dimension every side of the boundary.

show-area (boolean) - Default: false: Calculate and display the total area inside the boundary.

show-angle (array) - Default: (false, false): (show-boolean, flip-boolean). Calculates and draws internal degrees for every corner.

layer-styles (dictionary) - Default: (:): Defines the visual style for custom layers. E.g., ("Setbacks": (stroke: red, dash: "dashed")).

Automated CAD Schedules

plot-summary-table(..plots): Takes one or more trace dictionaries (or an array of subdivisions) and generates a formatted Typst table showing the Plot Name, Perimeter, and Total Area.

peg-schedule(plot-data): Extracts every single turning point from the main boundary layer and generates a surveyor's Corner Coordinate table showing the exact Easting (X) and Northing (Y) coordinates for stakeout.

Arch (API)

A

```
A(
  angle: angle,
  len: float,
  ..args: any
)
```

Tracing Command: Move at Angle

Parameters

- **angle** `angle`
  Direction.
- **len** `float`
  Distance.
- **..args** `any`

C

```
C(
  ..args: any
)
```

Tracing Command: Close the boundary back to the start coordinate.

Parameters

- **..args** `any`

D

```
D(
  len: float,
  ..args: any
)
```

Tracing Command: Move Down

Parameters

- **len** `float`
  Distance to move.
- **..args** `any`

DropX

```
DropX(
  p1: any,
  p2: any,
  ..args: any
)
```

Tracing Command: Drop in X direction until intersecting the line p1-p2

Parameters

- **p1** `any`
- **p2** `any`
- **..args** `any`

DropY

```
DropY(
  p1: any,
  p2: any,
  ..args: any
)
```

Tracing Command: Drop in Y direction until intersecting the line p1-p2

Parameters

- **p1** `any`
- **p2** `any`
- **..args** `any`

GoToMark

```
GoToMark(
  name: any,
  ..args: any
)
```

Tracing Command: Draw a wall directly to a previously saved mark

Parameters

- **name** `any`
- **..args** `any`

Home

```
Home( )
```

Tracing Command: Teleport to (0,0) relative to trace start

JA

```
JA(
  angle: any,
  len: any
)
```

Tracing Command: Jump at Angle (Lift pen)

Parameters

- **angle** `any`
- **len** `any`

JD

```
JD(
  len: any
)
```

Tracing Command: Jump Down (Lift pen)

Parameters

- **len** `any`

JDropX

```
JDropX(
  p1: any,
  p2: any
)
```

Tracing Command: Jump drop X (intersect without drawing)

Parameters

- **p1** `any`
- **p2** `any`

JDropY

```
JDropY(
  p1: any,
  p2: any
)
```

Tracing Command: Jump drop Y (intersect without drawing)

Parameters

- **p1** `any`
- **p2** `any`

JL

```
JL(
  len: any
)
```

Tracing Command: Jump Left (Lift pen)

Parameters

- **len** `any`

JR

```
JR(
  len: any
)
```

Tracing Command: Jump Right (Lift pen)

Parameters

- **len** `any`

JU

```
JU(
  len: any
)
```

Tracing Command: Jump Up (Lift pen)

Parameters

- **len** `any`

JumpToMark

```
JumpToMark(
  name: any
)
```

Tracing Command: Lift the pen and teleport to a saved mark

Parameters

- **name** `any`

L

```
L(
  len: float,
  ..args: any
)
```

Tracing Command: Move Left

Parameters

- **len** `float`

  Distance to move.

- **..args** `any`

Mark

```
Mark(
  name: any
)
```

Tracing Command: Save coordinate into memory

Parameters

- **name** `any`

R

```
R(
  len: float,
  ..args: any
)
```

Tracing Command: Move Right

Parameters

- **len** `float`

  Distance to move.

- **..args** `any`

U

```
U(
  len: float,
  ..args: any
)
```

Tracing Command: Move Up

Parameters

- **len** `float`
  Distance to move.
- **..args** `any`

arch-canvas

```
arch-canvas(
  body: content,
  scale: length
)
```

A dedicated canvas wrapper for interior architecture and floor plans.

Parameters

- **body** `content`
  The drawing code.
- **scale** `length`
  The scale of the canvas.

callout

```
callout(
  target: coordinate,
  text-pos: coordinate,
  label: content string,
  shoulder: float,
  stroke: stroke,
  size: length
)
```

Draws a drafting callout (leader line with a shoulder) to point at details.

Parameters

- **target** `coordinate`
  The exact coordinate you are pointing at.
- **text-pos** `coordinate`
  Where the text and the shoulder line begin.
- **label** `content` `string`
  The text to display.
- **shoulder** `float`
  The length of the flat landing line.
- **stroke** `stroke`
  Line style.
- **size** `length`
  Text size.

cm

```
cm(
  centimeters: float
)
```

Converts Centimeters into Decimal Feet.

Parameters

- **centimeters** `float`

  The measurement in centimeters.

column

```
column(
  pos: coordinate,
  size: float,
  sides: int,
  fill: color,
  align: string,
  angle: angle,
  shift: array float
)
```

Draws an N-sided structural column.

Parameters

- **pos** `coordinate`

  Coordinate location.

- **size** `float`

  Diameter or width of the column.

- **sides** `int`
  4 (Square), 6 (Hexagon), 30+ (Circle).
- **fill** `color`
  Fill color.
- **align** `string`
  Alignment shift ("center", "left", "right").
- **angle** `angle`
  Rotation angle.
- **shift** `array` `float`
  Manual X/Y shift.

create-grid

```
create-grid(
  x-grids: dictionary,
  y-grids: dictionary
)
```

GENERATE GRID: Takes a dictionary of X positions and Y positions
Returns a function 'get-node(x-key, y-key)'

Parameters

- **x-grids** `dictionary`
  X coordinates.
- **y-grids** `dictionary`
  Y coordinates.

dim

```
dim(
  from: coordinate,
  to: coordinate,
  offset: float,
  label: auto content,
  show-line: boolean,
  size: length,
  shift: array,
  units: string
)
```

Creates an external dimension line between two points.

Parameters

- **from** `coordinate`
  Start point.
- **to** `coordinate`
  End point.
- **offset** `float`
  Distance to pull the dimension line away.
- **label** `auto` `content`
  Optional manual label override.
- **show-line** `boolean`
  Show the physical line.
- **size** `length`
  Text size.
- **shift** `array`
  Array to pull the endpoints inwards (e.g. `(0.5, 0.5)`).

- **units** `string`
  "imperial" or "metric".

door

```
door(
  from: coordinate,
  to: coordinate,
  dist: float,
  width: float,
  hinge: string,
  swing: string,
  open: angle,
  thickness: float,
  align: string,
  label: content string none,
  label-offset: array
)
```

Places a parametric single door along a wall segment.

Parameters

- **from** `coordinate`
  The starting point of the wall segment.
- **to** `coordinate`
  The ending point of the wall segment.
- **dist** `float`
  Distance from the `from` point to the center of the door.
- **width** `float`
  The width of the door opening.

- **hinge** `string`
  Which side the hinge is on ("left" or "right").
- **swing** `string`
  Which way the door opens ("in" or "out").
- **open** `angle`
  The angle the door leaf is open.
- **thickness** `float`
  Thickness of the wall to match.
- **align** `string`
  Wall alignment logic.
- **label** `content` `string` `none`
  Optional text to display on the door.
- **label-offset** `array`
  An optional (X, Y) array to nudge the label.

double-door

```
double-door(
  from: coordinate,
  to: coordinate,
  dist: float,
  width: float,
  swing: string,
  open: angle,
  thickness: float,
  align: string,
  label: content string none
)
```

Places a parametric double door along a wall segment.

Parameters

- **`from`** `coordinate`
  The starting point of the wall segment.
- **`to`** `coordinate`
  The ending point of the wall segment.
- **`dist`** `float`
  Distance from the `from` point to the center of the door.
- **`width`** `float`
  The total width of the double door opening.
- **`swing`** `string`
  Which way the doors open ("in" or "out").
- **`open`** `angle`
  The angle the door leaves are open.
- **`thickness`** `float`
  Thickness of the wall to match.
- **`align`** `string`
  Wall alignment logic.
- **`label`** `content` `string` `none`
  Optional text to display between the leaves.

drafting-grid

```
drafting-grid(
  width: float,
  height: float,
  step: float,
  stroke: stroke,
  text-size: length
)
```

Draws a light background grid and labels every intersection.

Parameters

- **`width`** `float`
  Canvas width.
- **`height`** `float`
  Canvas height.
- **`step`** `float`
  Grid spacing.
- **`stroke`** `stroke`
  Line style.
- **`text-size`** `length`
  Label text size.

drafting-ruler

```
drafting-ruler(
  width: float,
  height: float,
  step: float,
  stroke: stroke,
  text-size: length,
  tick: float
)
```

Draws CAD-style coordinate rulers strictly on the Left and Top edges.

Parameters

- **width** `float`
  Canvas width.
- **height** `float`
  Canvas height.
- **step** `float`
  Tick spacing.
- **stroke** `stroke`
  Line style.
- **text-size** `length`
  Label text size.
- **tick** `float`
  Length of the tick marks.

draw-walls

```
draw-walls(
  wall-data: dictionary array,
  default-thickness: float,
  border-width: float,
  wall-color: color
)
```

Renders your traced walls with dynamic thickness and clean corner joins.

Parameters

- **wall-data** `dictionary` `array`
  The data returned from trace-walls.

- **`default-thickness`** `float`
  The default thickness for all walls.
- **`border-width`** `float`
  The thickness of the outer stroke lines.
- **`wall-color`** `color`
  The interior fill color of the walls.

format-area

```
format-area(
  sq-feet: float,
  unit-system: string
)
```

Formats the raw square feet into display text.

Parameters

- **`sq-feet`** `float`
  The raw calculated area in square feet.
- **`unit-system`** `string`
  "imperial" or "metric".

format-dist

```
format-dist(
  dist-in-feet: float,
  unit-system: string
)
```

Takes a distance in feet and formats it as text.

Parameters

- **`dist-in-feet`** `float`

  The raw distance in decimal feet.

- **`unit-system`** `string`

  "metric" (m/cm) or "imperial" (feet/inches).

ft

```
ft(
  feet: int float,
  inches: int float
)
```

Helper to convert Feet + Inches to Decimal Feet.
Usage: `ft(5, 6)` -> `5.5`

Parameters

- **`feet`** `int` `float`

  The feet value.

- **`inches`** `int` `float`

  The inches value.

get-align-dy

```
get-align-dy(
  align: string,
  thickness: float
)
```

Internal helper: Calculates the perpendicular shift based on alignment

Parameters

- **align** `string`
  "left", "right", or "center"
- **thickness** `float`
  Wall thickness

get-angle

```
get-angle(
  v: array
)
```

Internal helper: Calculate angle manually using standard Typst math

Parameters

- **v** `array`
  The vector.

get-area

```
get-area(
  vertices: array
)
```

Calculates area for a given array of vertices using the Shoelace formula.

Parameters

- **vertices** `array`
  List of coordinates.

get-length

```
get-length(
  p1: coordinate,
  p2: coordinate
)
```

Calculate the exact distance between any two coordinates.

Parameters

- **p1** `coordinate`
  First point.
- **p2** `coordinate`
  Second point.

get-pts

```
get-pts(
  wall-list: array
)
```

Helper to extract an array of plain coordinates from a list of wall dictionaries.

Parameters

- **`wall-list`** `array`
  Sliced array of walls.

i-dim

```
i-dim(
  from: coordinate,
  to: coordinate,
  thick-start: float,
  thick-end: float,
  offset: float,
  size: length
)
```

ADVANCED INTERNAL DIMENSION HELPER

Parameters

- **`from`** `coordinate`
  Start point.
- **`to`** `coordinate`
  End point.
- **`thick-start`** `float`
  How much to shrink from the 'from' point (e.g. 0.75)

- **thick-end** `float`
  How much to shrink from the 'to' point (e.g. 0.375)
- **offset** `float`
  Distance to move the dimension line sideways
- **size** `length`
  Text size.

inset-polygon

```
inset-polygon(
  pts: any,
  widths: any
)
```

Internal math helper to offset a polygon inward by given widths.

Parameters

- **pts** `any`
- **widths** `any`

intersect-lines

```
intersect-lines(
  p1: any,
  v1: any,
  p2: any,
  v2: any
)
```

Internal math helper to find the intersection of two vector lines.

Parameters

- **p1** `any`
- **v1** `any`
- **p2** `any`
- **v2** `any`

m

```
m(
  meters: float
)
```

Converts Meters into Decimal Feet.
Usage: `m(2)` -> `6.561...` (Allows you to type metric into your feet-based grid)

Parameters

- **meters** `float`
  The measurement in meters.

move

```
move(
  pt: coordinate,
  dx: float,
  dy: float
)
```

RELATIVE MOVER: Moves a point relative to a grid intersection
Usage: `rel("A-1", dx: 0.5, dy: -0.75)`

Parameters

- **pt** `coordinate`
  Base coordinate.
- **dx** `float`
  X shift.
- **dy** `float`
  Y shift.

opening

```
opening(
  from: coordinate,
  to: coordinate,
  dist: float,
  width: float,
  style: string,
  thickness: float,
  align: string,
  shift: float,
  label: content string none,
  label-offset: float,
  stroke: stroke
)
```

Creates a cased opening or pass-through in a wall without a door leaf.

Parameters

- **`from`** `coordinate`
  The starting point of the wall segment.
- **`to`** `coordinate`
  The ending point of the wall segment.
- **`dist`** `float`
  Distance from the start point to the center.
- **`width`** `float`
  The width of the opening.
- **`style`** `string`
  "empty", "line", "rect", or "cross".
- **`thickness`** `float`
  Wall thickness.
- **`align`** `string`
  Wall alignment logic.
- **`shift`** `float`
  Vertical shift for the inner style elements.
- **`label`** `content` `string` `none`
  Optional text.
- **`label-offset`** `float`
  Distance to nudge the label from center.
- **`stroke`** `stroke`
  The stroke style of the internal graphics.

ots

```
ots(
  pts: array,
  name: string,
  wall-widths: array none,
  show-area: boolean,
  show-dim: boolean,
  stroke: stroke,
  size: length,
  shift: array,
  units: string,
  fill: color
)
```

Takes an array of points and draws an "X" across them to denote a shaft or void.

Parameters

- **pts** `array`
  Coordinates defining the boundary.
- **name** `string`
  Label.
- **wall-widths** `array` `none`
  Shrink boundary inward.
- **show-area** `boolean`
  Show area.
- **show-dim** `boolean`
  Show dimensions.
- **stroke** `stroke`
  Line style.

- **size** `length`
  Text size.
- **shift** `array`
  Offset for the text.
- **units** `string`
  "imperial" or "metric".
- **fill** `color`
  Background fill.

poly-room

```
poly-room(
  pts: array,
  name: string content,
  wall-widths: array none,
  show-area: boolean,
  show-dim: boolean,
  stroke: stroke,
  shift: array,
  size: length,
  fill: color,
  units: string
)
```

Automatically generates, insets, and dimensions a multi-sided room.

Parameters

- **pts** `array`
  An array of coordinates.

- **name** `string` `content`
  The label for the room.
- **wall-widths** `array` `none`
  Array of wall thicknesses to inset the boundary.
- **show-area** `boolean`
  Displays calculated area.
- **show-dim** `boolean`
  Displays Width x Height.
- **stroke** `stroke`
  Boundary line style.
- **shift** `array`
  Offset for the text label.
- **size** `length`
  Text size.
- **fill** `color`
  Room fill color.
- **units** `string`
  "imperial" or "metric".

print-marks

```
print-marks(
  trace-data: dictionary,
  size: length,
  color: color,
  offset: array
)
```

Overlays crosshairs and text labels on all saved Marks for debugging.

Parameters

- **`trace-data`** `dictionary`
  The result object from trace-walls.
- **`size`** `length`
  Text size.
- **`color`** `color`
  Label color.
- **`offset`** `array`
  X/Y shift for the label.

stairs

```
stairs(
  pos: coordinate,
  steps: int array,
  width: float,
  run: float,
  angle: angle,
  turn: string array,
  split-landing: boolean
)
```

Generates a parametric staircase (Straight, L-Shape, or U-Shape).

Parameters

- **pos** `coordinate`
  The starting bottom-left coordinate.
- **steps** `int` `array`
  Number of steps, or array for multiple runs separated by landings (e.g. `(5, 5)`).
- **width** `float`
  The width of the staircase.
- **run** `float`
  The depth of each individual tread.
- **angle** `angle`
  Global rotation of the entire staircase.
- **turn** `string` `array`
  Direction the landing turns ("right" or "left").
- **split-landing** `boolean`
  True to draw a diagonal winder line on landings.

trace-walls

```
trace-walls(
  ops: array,
  start: coordinate,
  ..global-args: arguments
)
```

Continuous tracing engine for drafting walls and boundaries.
Returns a dictionary with `walls` (the line data) and `anchors` (saved marks).

Parameters

- **ops** `array`
  A list of tracing commands (R, L, U, D, Mark, etc.).
- **start** `coordinate`
  The starting position.
- **..global-args** `arguments`
  Default parameters applied to all walls in this trace.

wall-shape

```
wall-shape(
  from: any,
  to: any,
  thickness: any,
  shift: any,
  join: any,
  stroke: any,
  fill: any,
  ext-start: any,
  ext-end: any
)
```

Internal helper: Renders an individual wall segment.

Parameters

- **from** `any`
- **to** `any`
- **thickness** `any`
- **shift** `any`
- **join** `any`

- **stroke** `any`
- **fill** `any`
- **ext-start** `any`
- **ext-end** `any`

window

```
window(
  wall-start: coordinate,
  wall-end: coordinate,
  dist: float,
  width: float,
  thickness: float,
  align: string,
  label: content string none,
  label-offset: float
)
```

Places a window along a wall, drawing glass panes and erasing the wall line.

Parameters

- **wall-start** `coordinate`
  The starting point of the wall.
- **wall-end** `coordinate`
  The ending point of the wall.
- **dist** `float`
  Distance from the start point to the center of the window.

- **`width`** `float`
  The width of the window.
- **`thickness`** `float`
  The thickness of the wall.
- **`align`** `string`
  Wall alignment logic.
- **`label`** `content` `string` `none`
  Optional text tag.
- **`label-offset`** `float`
  Distance to shift the label away from the glass.

bricks-fill

Built-in material hatch pattern: Bricks

hatch-fill

Built-in material hatch pattern: Diagonal Lines

grass-fill

Built-in material hatch pattern: Grass